

Universidad Católica de Temuco

Ingeniería Civil en Informática

Reimplementación compatible de OpenStack Placement en un lenguaje compilado

Etapla 2: análisis, diseño y línea base

Estudiante: Paolo Castañon Barrera

Profesor guía: Alejandro Mellado Gatica

Curso: Trabajo de Título (INFO-1197)

Fecha: 2 de octubre de 2026

Temuco, Chile

Índice

1	Introducción	2
2	Definición del problema	2
2.1	Contexto	2
2.2	Problema identificado	2
2.3	Relevancia e impacto	3
2.4	Partes interesadas y percepción del problema	3
2.5	Alcance y límites	3
3	Objetivos	3
3.1	Objetivo general	3
3.2	Objetivos específicos	4
3.3	Verificación SMART	4
3.4	Matriz de trazabilidad problema–objetivos	5
4	Análisis de requerimientos	5
4.1	Base técnica y arquitectura del servicio	5
4.2	Selección del servicio y del lenguaje	5
4.3	Entorno de despliegue: DevStack versus instalación manual	6
4.4	Contrato API y microversiones	7
4.5	Modelo de datos y consistencia	7
4.6	Interacción Nova–Placement	9
4.7	Requerimientos funcionales	10
4.8	Requerimientos no funcionales	10
4.9	Arquitectura del reemplazo Go y criterios de aceptación	11
4.10	Notación y trazabilidad de la calidad del diseño	11
4.11	Decisiones técnicas y alternativas	12
4.12	Riesgos y mitigaciones	13
5	Planificación y backlog	13
5.1	Épicas	13
5.2	Backlog prioritario y estado del Sprint 2	14
5.3	Inventario de diseño y comparación con el backlog completo	14
5.4	Repositorio, decisiones y cambios trazables	15
5.5	Refinamientos y desviaciones respecto del plan inicial	17
5.6	Matriz de evidencia para los criterios técnicos de Etapa 2	18
5.7	Plan por sprints e hitos	18
6	Conclusiones de la Etapa 2	19
	Referencias	19

1 Introducción

OpenStack Placement registra capacidad y uso de recursos para apoyar la selección de destinos de cómputo. Nova consulta este servicio durante la planificación y mantiene asignaciones asociadas a cada instancia. Su API HTTP ofrece un límite de integración adecuado para estudiar una reimplementación en un lenguaje compilado [1, 2].

Este informe corresponde al trabajo de Paolo Castañón Barrera y concentra el análisis en Placement. Se conserva la organización del informe de Etapa 1: problema, objetivos, requerimientos, planificación y conclusiones. La Etapa 2 desarrolla esos apartados con arquitectura, contrato de API, modelo de datos, decisiones de diseño y evidencia del laboratorio.

La propuesta utiliza Go y evalúa dos dimensiones: compatibilidad funcional y comportamiento bajo carga. La mejora de rendimiento es una hipótesis que debe medirse. En esta etapa se verificó el servicio original; la implementación Go y los benchmarks comparativos corresponden a las etapas siguientes.

El entorno es compartido con el proyecto de Glance, pero el objeto de estudio de este documento es Placement. Los resultados comunes de infraestructura se identifican como tales y no se presentan como mediciones exclusivas del módulo ni como atribución individual de la ejecución.

2 Definición del problema

2.1 Contexto

El despliegue de una instancia requiere coordinar Nova, Keystone, Neutron, Glance y Placement. En el flujo de planificación, Placement entrega candidatos que satisfacen recursos y restricciones; Nova decide el destino y registra el consumo. El servicio combina semántica HTTP, negociación de microversiones y reglas de consistencia persistente [1, 2].

La línea base se obtuvo en una instalación DevStack de un nodo. Esta configuración permite observar llamadas reales y repetir escenarios controlados. No representa un despliegue de producción ni una evaluación de escalado multinodo.

2.2 Problema identificado

Cambiar el lenguaje de implementación puede alterar códigos HTTP, formatos JSON, errores, filtrado de candidatos o control de concurrencia. Un servidor que responde a rutas conocidas no demuestra por sí solo que Nova pueda utilizarlo. La dificultad técnica es conservar el contrato observable del subconjunto necesario y medir sus costos con una comparación repetible.

La pregunta de investigación es: *¿es posible reimplementar el alcance seleccionado de Placement en Go, conservando la integración con Nova, y qué diferencias de latencia, throughput y uso de recursos se observan frente al servicio original?*

2.3 Relevancia e impacto

La contribución esperada es una especificación verificable y una implementación experimental integrada. El trabajo puede identificar costos de procesamiento HTTP, acceso a datos y búsqueda de candidatos; también puede mostrar regresiones o límites de compatibilidad. La validez del resultado depende de reportar ambos tipos de hallazgo.

2.4 Partes interesadas y percepción del problema

Cuadro 1: Partes interesadas del proyecto Placement.

Parte interesada	Necesidad o interés
Estudiante responsable	Paolo Castañon Barrera: delimitar, diseñar e implementar Placement y sustentar las conclusiones con pruebas.
Profesor guía	Revisar coherencia entre problema, alcance, evidencia y resultados del trabajo de título.
Consumidores de la API	Nova y clientes de OpenStack: conservar el contrato HTTP y la consistencia del estado.
Equipo del laboratorio	Reconstruir el entorno común y aislar la sustitución de cada servicio.

2.5 Alcance y límites

Se incluyen proveedores de recursos, inventarios, clases de recursos, traits, allocations y candidatos utilizados en el escenario de un nodo. El análisis contempla autenticación Keystone, persistencia y generaciones para resolver conflictos.

El servidor original anuncia microversiones 1.0 a 1.39. La propuesta inicial selecciona 1.0, 1.2, 1.6, 1.10, 1.28 y 1.36 según la operación; esto no equivale a implementar todas las microversiones intermedias ni toda la API. `POST/reshaper`, migraciones complejas de inventario y topologías no exigidas por las pruebas quedan fuera del alcance inicial.

Glance se mantiene como dependencia original al probar la sustitución de Placement. No se reimplementan Nova, Keystone, Neutron ni el sistema de virtualización. Tampoco se promete sustituir una base de datos existente sin un procedimiento de migración.

3 Objetivos

3.1 Objetivo general

Diseñar, implementar y evaluar una reimplementación compatible del subconjunto seleccionado de OpenStack Placement en Go, integrada con Nova en un laboratorio reproducible, comparando su funcionamiento, rendimiento y uso de recursos con el servicio original.

3.2 Objetivos específicos

Los identificadores OE1–OE5 conservan la trazabilidad de la planificación común; su redacción aquí se aplica a Placement. El porcentaje del hito OE3 corresponde al proyecto conjunto y no se utiliza como medida de avance individual.

- OE1.** Analizar arquitectura, API, dependencias, datos e interacción Nova–Placement antes del 2 de octubre de 2026.
- OE2.** Disponer de un entorno reproducible y una línea base funcional del servicio original antes del 2 de octubre de 2026.
- OE3.** Implementar al menos el 50% del alcance comprometido, con pruebas preliminares, antes del 30 de octubre de 2026.
- OE4.** Integrar y estabilizar el alcance de Placement con Nova antes del 20 de noviembre de 2026.
- OE5.** Comparar compatibilidad, latencia, throughput, CPU, memoria y errores, y presentar conclusiones antes del 4 de diciembre de 2026.

3.3 Verificación SMART

Cuadro 2: Objetivos, indicadores y estado al cierre de Sprint 2.

ID	Indicador y evidencia	Plazo	Estado
OE1	Arquitectura y rutas del alcance documentadas; modelo auditado y tráfico Nova registrado.	02-10-2026	Cumplido en S2.
OE2	Laboratorio instalado; smoke test y baseline ejecutados; instancia en ACTIVE y limpieza.	02-10-2026	Cumplido en S2.
OE3	Al menos 50% del alcance priorizado implementado y probado en Go.	30-10-2026	Planificado.
OE4	Prueba de sustitución con Nova y casos críticos sin regresiones.	20-11-2026	Planificado.
OE5	Dataset comparable, repeticiones y análisis de métricas y limitaciones.	04-12-2026	Planificado.

La medición de OE1 se refiere al alcance documentado para Sprint 2. Los endpoints definidos pero aún no ejercitados quedan señalados para ampliar sus fixtures; no se infiere cobertura experimental total de la API.

3.4 Matriz de trazabilidad problema–objetivos

Cuadro 3: Trazabilidad del problema Placement.

Aspecto del problema	Objetivo	Resultado verificable
Contrato y versiones desconocidos	OE1	Inventario de métodos, rutas y microversiones; llamadas reales de Nova.
Instalación y pruebas difíciles de repetir	OE2	Terraform, Ansible, versiones fijadas y scripts versionados.
Consistencia y equivalencia del reemplazo	OE3/OE4	Transacciones, conflictos de generación y pruebas diferenciales/integradas.
Hipótesis de eficiencia sin medir	OE5	Comparación controlada de latencia, throughput y recursos.

4 Análisis de requerimientos

4.1 Base técnica y arquitectura del servicio

En el código desplegado se identifican entrada WSGI, middleware de autenticación, políticas, negociación de microversión, handlers HTTP, objetos de dominio y persistencia SQLAlchemy. Las generaciones de providers y consumers forman parte de las reglas de actualización. Estas capas se documentaron a partir del código del commit fijado y se contrastaron con API y tablas reales [4].

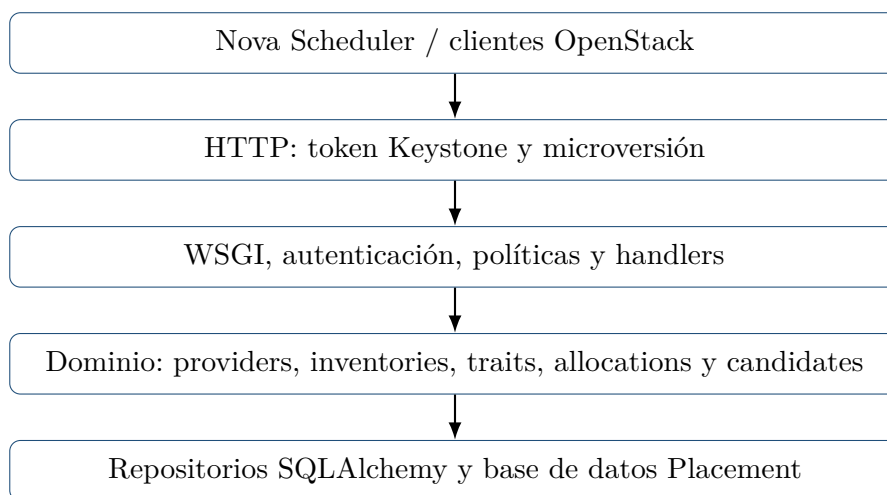


Figura 1: Componentes del servicio original y límite HTTP de integración.

4.2 Selección del servicio y del lenguaje

Placement se selecciona por su función en la planificación de Nova y por disponer de un contrato HTTP que permite sustituir el proceso del servicio. La selección no implica que su comportamiento sea simple: generaciones, filtros y capacidad disponible deben conservarse.

Se propone Go por su biblioteca HTTP, compilación a binario y soporte de concurrencia. La valoración siguiente es una decisión de diseño del proyecto; no constituye un benchmark entre lenguajes.

Cuadro 4: Alternativas de lenguaje para la implementación Placement.

Criterio	Go	Rust	C++
Desarrollo HTTP	Biblioteca estándar y ecosistema adecuados.	Ecosistema adecuado; mayor curva inicial.	Requiere integrar bibliotecas.
Memoria y concurrencia	Gestión automática; goroutines.	Control estricto de propiedad.	Mayor responsabilidad manual.
Riesgo en el plazo	Moderado.	Mayor costo inicial estimado.	Mayor costo de integración estimado.
Decisión	Seleccionado.	Alternativa.	No priorizado.

4.3 Entorno de despliegue: DevStack versus instalación manual

DevStack automatiza un entorno de desarrollo y pruebas [3]. Frente a una instalación manual, reduce la configuración inicial y permite concentrar el trabajo en compatibilidad. La instalación manual brinda control detallado, pero exige documentar más pasos y resolver dependencias por separado.

El laboratorio común utiliza Terraform con libvirt y Ansible. La VM `tt-openstack-lab` tiene Ubuntu 24.04.5, 4 vCPU, 8 GiB de RAM y disco de 55 GiB. Nova usa QEMU emulado. Se verificaron Keystone, Nova, Placement, Glance, Neutron, MySQL y RabbitMQ, y una segunda ejecución del aprovisionamiento terminó sin reinstalar DevStack.

Cuadro 5: Versiones registradas en la línea base.

Componente	Versión o commit
DevStack	<code>stable/2026.2;</code> <code>0fb9685c1bf22cd0b4d576d7a74f64195ad934fe.</code>
Placement	<code>f4a89d3803cb23df22b8714569889ab39c918c34.</code>
Nova	<code>247ea5d774305b688997f8df81c011b249eb793c.</code>
Python / MySQL	3.12.3 / 8.0.46.
OpenStackClient	10.3.0.

La configuración y un parche de inicialización Geneve están versionados. Las dependencias secundarias no están fijadas para reproducción bit a bit. Al cierre del trabajo la VM quedó apagada por solicitud del usuario, conservando sus discos; este informe usa evidencia obtenida antes del apagado.

4.4 Contrato API y microversiones

La autenticación utiliza tokens Keystone. El contrato debe conservar entradas, respuestas, errores y headers relevantes para cada microversión. El cuadro distingue llamadas ejercitadas de operaciones todavía pendientes de captura.

Cuadro 6: Operaciones Placement priorizadas y cobertura de la línea base.

Método y ruta	Versión	Evidencia o estado
GET /	Anunciada 1.0–1.39	Descubrimiento del servidor original.
GET/POST /resource_providers	1.0	Listado y creación probados.
GET/DELETE /resource_providers/{uuid}	1.0	Consulta, eliminación y 404 probados.
PUT /resource_providers/{uuid}	1.0	Diseñado; fixture pendiente.
GET/PUT /resource_providers/{uuid}/inventories	1.0	Consulta y reemplazo de inventarios probados.
GET /resource_classes	1.2	Listado probado; consulta por nombre pendiente.
GET /resource_providers/{uuid}/traits	1.6	Consulta probada; asociación y listado global pendientes.
GET /allocation_candidates	1.10 / 1.36	Baseline y petición real de Nova, HTTP 200.
GET /allocations/{consumer}	1.28	Petición real de Nova, HTTP 200.
PUT /allocations/{consumer}	1.36	Reclamación real de Nova, HTTP 204.
DELETE /allocations/{consumer}	1.0	Liberación real de Nova, HTTP 204.

Para la implementación se requieren casos negativos de entradas inválidas, conflictos de generación y autorización. Un error 404 ya quedó registrado en la línea base, pero no se considera suficiente para aceptar todo el manejo de errores. Las clases personalizadas son condicionales; `resaper` queda excluido inicialmente.

4.5 Modelo de datos y consistencia

La auditoría contrastó las tablas MySQL con los modelos SQLAlchemy del servicio fijado. Algunas relaciones son lógicas u ORM y no corresponden a una FK SQL declarada; el diseño evita asumir restricciones que la captura de tablas no demuestra.

Cuadro 7: Entidades centrales y restricciones relevantes.

Entidad	Datos centrales	Regla de diseño
ResourceProvider	ID, UUID y nombre únicos; generación; relaciones padre/raíz.	Identidad estable y actualización con generación.
Inventory	Provider, clase, total, reservado, límites, paso y ratio.	Unicidad provider/clase y validación de capacidad.
ResourceClass / Trait	Identificador y nombre; asociaciones provider/trait.	Preservar clases y rasgos usados por Nova.
Consumer	UUID, proyecto, usuario, generación y tipo.	Asociar consumo y controlar concurrencia.
Allocation	Provider, consumer, clase y cantidad usada.	Operaciones atómicas y consistencia entre uso e inventario.

Se decide usar un esquema propio en una base separada para Go y mantener compatibilidad por API. Esta decisión reduce acoplamiento con las migraciones Python, pero exige semillas y comparación del estado. Las transacciones deben comprobar capacidad y generaciones antes de confirmar allocations; las reglas concretas se validarán con fixtures diferenciales.

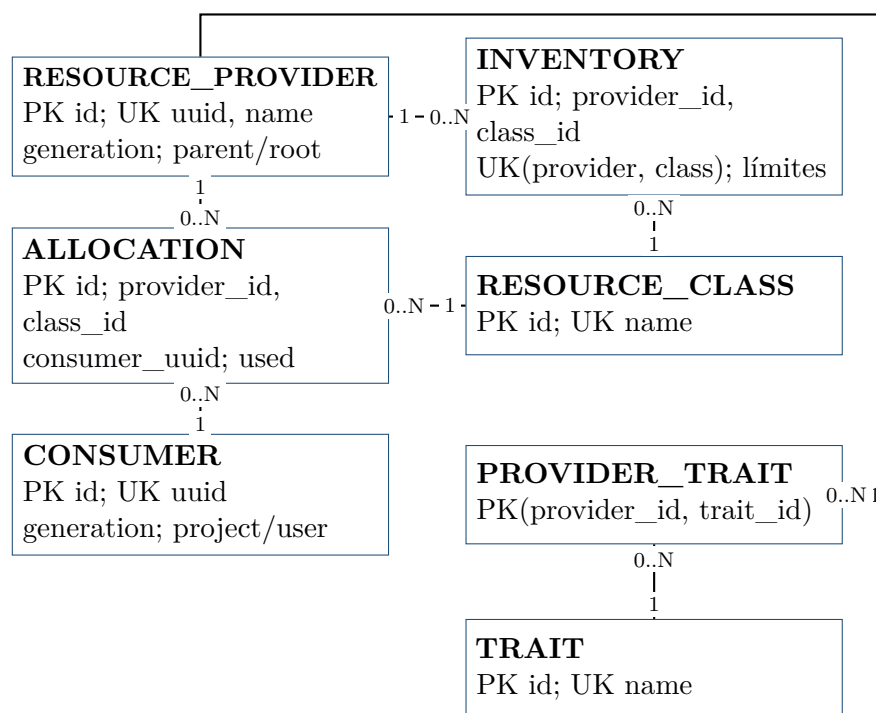


Figura 2: Modelo ER lógico del alcance Placement. PK: clave primaria; UK: unicidad. La tabla de atributos distingue las FK observadas de asociaciones lógicas.

El diseño Go conserva las relaciones lógicas de la figura 2, pero define sus propias migraciones. La transacción de claim debe leer generaciones y capacidad, validar la solicitud, actualizar allocations y generaciones, y confirmar o revertir el conjunto. Se compararán conflictos y rollback con el original antes de aceptar RF05/RF07.

4.6 Interacción Nova–Placement

La traza real del boot confirma búsqueda de candidatos, consulta de allocations, reclamación y liberación. La consulta de Nova solicitó `DISK_GB:1, MEMORY_MB:512, VCPU:1` y restricciones `OWNER_NOVA, !COMPUTE_STATUS_DISABLED`. Scheduler seleccionó `tt-openstack-lab` y la instancia alcanzó `ACTIVE`.

1. Nova solicita candidatos y recibe HTTP 200 con microversión 1.36.
2. Consulta las allocations del consumer y recibe HTTP 200 con 1.28.
3. Registra la reclamación con PUT y recibe HTTP 204 con 1.36.
4. La eliminación de la instancia libera allocations mediante DELETE, HTTP 204 con 1.0.

Los logs prueban método, ruta, resultado y microversión. No capturan el cuerpo completo de las solicitudes internas, por lo que aún se deben obtener fixtures específicos de esos cuerpos. El resultado `ACTIVE` valida la integración del original, no la de un reemplazo Go aún inexistente.

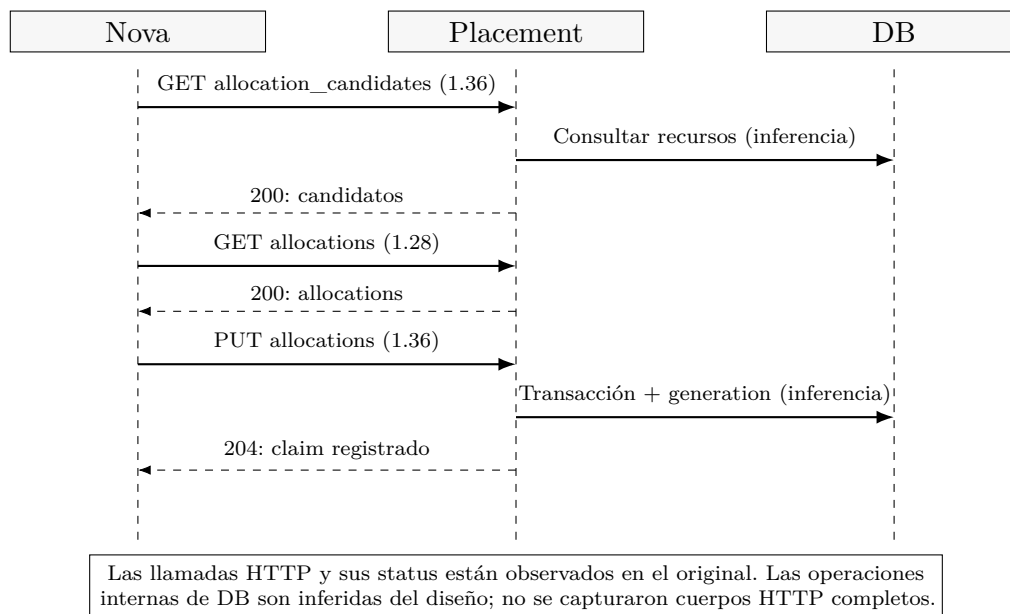


Figura 3: Secuencia Nova–Placement del boot original. Mensajes continuos: solicitudes; discontinuos: respuestas. La eliminación añade DELETE allocations 1.0, HTTP 204.

4.7 Requerimientos funcionales

Cuadro 8: Requerimientos aplicables al módulo Placement.

ID	Requerimiento	Verificación prevista
RF01	OpenStack de un nodo reproducible.	Instalación y smoke test; base validada en S2.
RF02	Rutas y microversiones del alcance.	Suite de contrato y comparación de headers/JSON.
RF03	Proveedores e inventarios compatibles.	CRUD, capacidad y conflictos.
RF04	Traits y clases requeridos.	Consultas, asociaciones y filtros priorizados.
RF05	Consulta y actualización de allocations.	Reclamación/liberación y generaciones.
RF06	Candidatos compatibles con Nova.	Comparación semántica de candidatos y boot.
RF07	Persistencia consistente.	Reinicio y transacciones concurrentes.
RF08	Autenticación y políticas necesarias.	Casos autorizados, 401 y 403.
RF09	Sustitución integrada con Nova.	Boot y eliminación usando Go.
RF11 / RF12	Observabilidad y automatización.	Logs saneados, scripts y métricas repetibles.

RF03–RF09 describen aceptación de la implementación futura. El análisis de Sprint 2 no equivale a tener estos requerimientos implementados en Go.

4.8 Requerimientos no funcionales

Cuadro 9: Atributos de calidad y criterios de verificación.

ID	Atributo	Criterio
RNF01	Compatibilidad	Comparar status, tipos, campos, errores, headers estables y efectos persistidos.
RNF02	Rendimiento	Latencia p50/p95/p99 y throughput con carga controlada.
RNF03	Recursos	CPU y memoria del proceso en reposo y carga.
RNF04	Fiabilidad	Repetir escenarios y registrar tasa de error.
RNF05	Reproducibilidad	Versiones, configuración, comandos y dataset documentados.
RNF06	Mantenibilidad	Capas separadas, pruebas y commits trazables.
RNF07	Seguridad	Conservar autenticación y autorización; excluir secretos de fixtures.

4.9 Arquitectura del reemplazo Go y criterios de aceptación

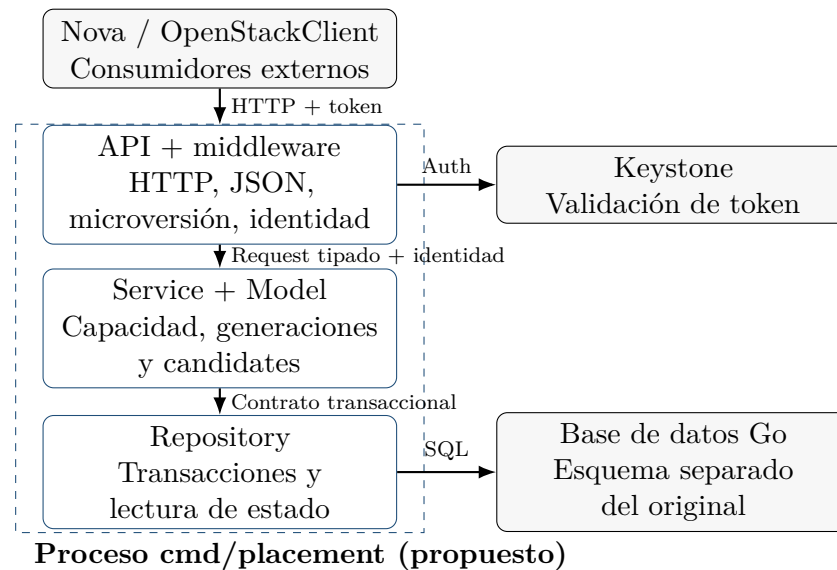


Figura 4: Vista de componentes del reemplazo Placement Go. Rectángulo discontinuo: límite del proceso; flechas: llamadas o dependencias.

El ejecutable propuesto es `cmd/placement`. Los paquetes internos separan API, servicio, modelo y repositorio. Los handlers negocian versiones y validan HTTP; el servicio aplica reglas de negocio; el repositorio controla transacciones. Configuración, middleware Keystone y logging se mantienen como componentes reutilizables.

Se aceptará la sustitución cuando el original y Go superen los casos comprometidos, Nova cree y elimine una instancia utilizando Go, y las comparaciones diferenciales no muestren diferencias semánticas relevantes. UUID, timestamps, request IDs y orden no garantizado podrán normalizarse sin ocultar cambios de capacidad o asignación.

Las pruebas abarcarán unidad, contrato, comparación diferencial, integración y escenario completo. La evaluación posterior usará misma VM, versiones, estado inicial, carga, warm-up y número de repeticiones. No se impone una mejora de velocidad como criterio de aceptación ni se presentan benchmarks de esta etapa.

4.10 Notación y trazabilidad de la calidad del diseño

La vista de componentes organiza un único proceso Go, sus responsabilidades, interfaces y dependencias externas según las abstracciones del modelo C4 [6]. Los modelos ER expresan entidades y cardinalidades; las secuencias usan participantes, líneas de vida, mensajes y respuestas siguiendo la notación básica de interacción de UML [7]. Cada figura distingue sistema original, propuesta Go e intercambios todavía inferidos. Las flechas se interpretan según su leyenda; una relación del dominio no implica una FK SQL.

La compatibilidad HTTP se verificará con método, status, media type, headers y efectos, atendiendo a la semántica de HTTP y al contrato específico de OpenStack [8]. Las matrices siguientes relacionan las decisiones con RF/RNF, objetivos y pruebas de aceptación. Describen prácticas de diseño y un plan de verificación; no constituyen certificación de cumplimiento de una norma ni resultados de pruebas Go.

Cuadro 10: Requerimientos, prácticas de diseño y verificaciones de aceptación.

Requerimiento	Práctica / artefacto	Prueba o criterio
RF02 / RNF01	API y contratos HTTP; secuencia de fig. 3.	Mismas rutas, headers, microversiones, status y errores; comparación diferencial.
RF03–RF06	Modelo ER (fig. 2); servicio separado de handlers.	Fixtures de inventario, traits, allocations y candidatos, con estado inicial equivalente.
RF05/RF07	Transacción y concurrencia optimista; ADR-01.	Claim concurrente, generación obsoleta y rollback; revisar capacidad y estado final.
RF08 / RNF07	Auth Keystone y políticas por operación; ADR-03.	Casos positivos y negativos 401/403; no guardar tokens ni omitir validación.
RF09 / RNF06	Proceso Go aislado (fig. 4); DB separada.	Sustitución controlada y boot/delete con Nova original; recuperación del original.
RNF05 / RF11/RF12	Versiones, configuración y logs con request ID.	Reprovisión repetida, fixtures saneados y scripts versionados.

4.11 Decisiones técnicas y alternativas

Cuadro 11: Decisiones de diseño Placement y sus consecuencias.

ADR	Contexto y alternativas	Decisión y razón	Consecuencia / aceptación
01	Persistencia: esquema original, propio o intermedio.	Esquema propio en DB separada; compatibilidad por HTTP evita depender de migraciones Python.	Semillas y migración explícitas; validar consistencia con transacciones y fixtures.
03	Auth: red privada, middleware Keystone o políticas completas.	Validar token y políticas mínimas del alcance; preservar RF08/RNF07 sin reimplementar Keystone.	Dependencia Keystone; comparar 401/403 y roles por ruta.
Módulo	Acoplar SQL a handlers o separar API, servicio y repositorio.	Separación de responsabilidades y request tipado con identidad validada.	Pruebas de dominio aisladas y traducción consistente de errores a HTTP.

4.12 Riesgos y mitigaciones

Cuadro 12: Riesgos del desarrollo Placement.

Riesgo	Nivel	Mitigación
Cobertura de microversiones mayor a la estimada	Alto	Priorizar llamadas reales de Nova y registrar exclusiones.
Conflictos o sobreasignación	Alto	Pruebas de generación, capacidad y atomicidad antes de optimizar.
Cambios en dependencias	Medio	Mantener SHA fijados y registrar paquetes secundarios.
Benchmark sesgado por QEMU o estado	Medio	Comparar procesos bajo la misma configuración y reportar límites del laboratorio.
Cuerpos Nova incompletos	Medio	Capturar fixtures saneados antes de aceptar la integración Go.
Metodología experimental pendiente	Medio	Completar B16 con carga, repeticiones y métricas antes de comparar rendimiento.

5 Planificación y backlog

5.1 Épicas

Se conservan las épicas del Excel vigente que son aplicables: E1 infraestructura y entorno; E2 análisis y compatibilidad; E3 reimplementación Placement; E4 integración OpenStack; E5 pruebas y benchmarking. E6 reimplementación Glance corresponde al informe independiente de ese módulo. E7 documentación incluye B21–B24 y registra arquitectura, reproducción, resultados e informe final.

5.2 Backlog prioritario y estado del Sprint 2

Los IDs del cuadro proceden del Excel vigente [5] y no del backlog preliminar del PDF de Etapa 1, cuya numeración fue refinada. Se conserva esa distinción para evitar asociar una evidencia al elemento equivocado.

Cuadro 13: Elementos de Sprint 2 relevantes para Placement.

ID	Elemento	SP	Estado
B01	Preparar VM de laboratorio (compartido).	3	Hecho: VM auditada.
B02	Instalar DevStack de un nodo (compartido).	5	Hecho: servicios verificados.
B03	Smoke tests del original (compartido).	5	Hecho: boot ACTIVE y limpieza.
B04	Analizar arquitectura interna Placement.	5	Hecho: componentes y datos.
B05	Documentar endpoints y microversiones.	5	Hecho: inventario y capturas.
B06	Registrar interacción Nova–Placement.	5	Hecho: logs reales del flujo.

Las tareas específicas de Placement suman 15 SP; las comunes, 13 SP. El Sprint 2 conjunto alcanzó 7/7 elementos y 33/33 SP al incluir B19 de Glance. Estas cifras describen la planificación compartida, no una estimación de productividad individual.

La evidencia está en `evidence/sprint2/placement/`, `evidence/sprint2/nova/` y `evidence/sprint2/tests/`. El registro de versiones es `infra/devstack/versions.lock`; la auditoría de tablas está en `evidence/sprint2/environment/database-schema-audit.txt`. Los documentos de contrato, modelo y flujo se encuentran en `docs/sprint2/`. La verificación `make sprint2-verify` aprobó los artefactos y evidencias requeridos.

5.3 Inventario de diseño y comparación con el backlog completo

El backlog vigente contiene 26 elementos y 185 SP: 7 Hecho, 1 En progreso y 18 Pendiente. El avance global por elementos terminados es $7/26 = 26,9\%$; por puntos aceptados es $33/185 = 17,8\%$. B21 no se cuenta como terminado ni se prorratan sus puntos. Estas métricas globales se distinguen del 100% de aceptación del Sprint 2 (7/7, 33/33 SP).

El conjunto de análisis y diseño central comprometido en S2 es el conjunto D_{S2} , formado por B04, B05, B06 y B19: 4 de 4 elementos terminados, 20 de 20 SP. En este informe, la evidencia propia del módulo se desarrolla por separado; el otro módulo aporta su expediente independiente.

Al revisar el backlog completo, B21 incluye documentación de arquitectura y decisiones como actividad continua de S3 y B16 diseña el protocolo experimental en S4. Sus estados formales siguen En progreso y Pendiente, respectivamente. Por ello, el 100% anterior describe exclusivamente el diseño central de S2: no acredita por sí solo el cierre de todas las actividades de diseño y documentación del proyecto. Los diagramas y matrices de esta revisión amplían el soporte de B21 sin modificar el estado del Excel.

Cuadro 14: Backlog completo: clasificación y estado formal al 2 de octubre de 2026.

ID	Elemento resumido	Naturaleza	SP	Sprint	Estado
B01	Preparar VM	Entorno	3	S2	Hecho
B02	Instalar DevStack	Entorno	5	S2	Hecho
B03	Smoke tests del original	Entorno	5	S2	Hecho
B04	Arquitectura Placement	Diseño S2	5	S2	Hecho
B05	Endpoints y microversiones	Diseño S2	5	S2	Hecho
B06	Interacción Nova–Placement	Diseño S2	5	S2	Hecho
B07	Providers e inventories	Implementación	8	S3	Pendiente
B08	Traits y resource classes	Implementación	8	S3	Pendiente
B09	Allocations	Implementación	8	S3	Pendiente
B10	Allocation candidates	Implementación	13	S3	Pendiente
B11	Persistencia Placement	Implementación	8	S3	Pendiente
B12	Autenticación OpenStack	Implementación	8	S3	Pendiente
B13	Sustituir Placement	Integración	8	S4	Pendiente
B14	Boot con Placement Go	Integración	8	S4	Pendiente
B15	Suite de integración	Integración	8	S4	Pendiente
B16	Metodología de benchmarking	Diseño experimental	5	S4	Pendiente
B17	Latencia y throughput	Medición	5	S5	Pendiente
B18	CPU y memoria	Medición	5	S5	Pendiente
B19	Análisis integral Glance	Diseño S2	5	S2	Hecho
B20	Catálogo de metadatos Glance	Implementación	13	S4	Pendiente
B21	Arquitectura y decisiones (continua)	Diseño y documentación	5	S3	En progreso
B22	Instalación, despliegue y pruebas	Documentación	5	S4	Pendiente
B23	Resultados comparativos	Análisis	8	S5	Pendiente
B24	Informe final	Entrega	8	S5	Pendiente
B25	Carga/descarga Glance	Implementación	13	S4	Pendiente
B26	Sustitución Glance y boot Nova	Integración	8	S5	Pendiente

5.4 Repositorio, decisiones y cambios trazables

Código, fuentes y evidencias se entregan en el [repositorio GitHub del proyecto](#). El historial Git conserva cambios con mensajes que identifican su propósito. La revisión base de los informes individuales es 06b6505; las fuentes editables están en `docs/sprint2/reports/` y los PDF en `output/pdf/`. El registro de diseño se apoya en `design-decisions.md`, `go-target-architecture.md`, los contratos API y las evidencias saneadas de `evidence/sprint2/`.

Cuadro 15: Cambios de infraestructura y evidencia registrados en Git.

Commit	Cambio registrado	Justificación y evidencia
6e0ea2d	Instalación DevStack mediante Ansible.	Aprovisionamiento repetible con versión fijada; infraestructura común y scripts en infra/.
b261278	Inicializar asignaciones Geneve antes de crear la red.	Corrección técnica necesaria para completar DevStack; parche versionado y posterior smoke test.
6abd728	Conservar MySQL durante las ejecuciones Ansible.	Evitar que una dependencia de cliente retire el servidor; reprovisión del laboratorio.
30491e9	Capturar baseline y verificaciones reales.	Servicio original, APIs, tablas, store y boot documentados en evidence/sprint2/.
5052749	Actualizar el libro derivado con evidencia.	Estados Hecho de S2 sustentados; original sin modificar.
203f001 / 06b6505	Separar informes Placement y Glance.	Un autor y módulo por entrega; conserva evidencia común y formato institucional.
6344e3f	Preservar evidencia aceptada al repetir provisión.	Evitar que una repetición sustituya las capturas ya usadas para aceptación.

5.5 Refinamientos y desviaciones respecto del plan inicial

Cuadro 16: Cambios del plan y justificación documental.

Referencia inicial	Estado de Etapa 2	Justificación y efecto
Backlog preliminar B01–B20 del informe de Etapa 1.	Excel vigente B01–B26; incorpora tareas técnicas y alcance Glance refinado.	Se usan los IDs del libro actual en todas las evidencias; no se equiparan automáticamente los IDs antiguos.
Entorno de un nodo previsto.	VM dedicada gestionada por Terraform/libvirt y Ansible; Nova usa QEMU emulado.	Aislar el laboratorio del host y reconstruir su configuración. La emulación limita la generalización del benchmark.
Reimplementación compatible por delimitar.	Placement por rutas/microversiones seleccionadas; Glance con Image API v2 y un filesystem.	El baseline y el tráfico del original acotan lo necesario. No se promete cobertura completa ni todos los stores.
Persistencia del reemplazo por resolver.	Esquema propio y DB aislada; Glance separa metadata de bytes.	ADR-01/ADR-02 reducen acoplamiento al original; exigen semillas, migración y reconciliación.
Informe conjunto de ambos módulos.	Dos informes individuales por instrucción de entrega.	Placement para Paolo Castañon Barrera y Glance para Juan Cardenas Uribe; las tareas comunes no se cuentan dos veces.
Cronograma del libro vigente.	Se conservan S2–S5 y sus estados; B20/B25 en S4 y B26 en S5.	La preparación de fixtures en S3 no adelanta formalmente las historias Glance. No se replanificó el Excel en esta revisión.

5.6 Matriz de evidencia para los criterios técnicos de Etapa 2

La pauta del curso denomina esta etapa *Diseño e Implementación Inicial*. Los tres criterios técnicos se verifican con el informe, el libro de seguimiento y el repositorio [9]. Sus pesos originales se conservan en el cuadro; no se convierten en una nota ni en un porcentaje de logro académico.

Cuadro 17: Localización de evidencia para los criterios técnicos de la pauta.

Criterio	Peso	Evidencia verificable y límite
Avance en el diseño	10%	Inventario de diseño S2 y backlog completo en esta sección. B04/B05/B06/B19: 4/4; B21 y B16 conservan estados del libro. No se afirma 100% del diseño global.
Calidad del diseño	55%	Figuras 4, 2 y 3; matriz RF/RNF-práctica-prueba; contratos y ADR del módulo. Diferencia explícita entre diseño y validación Go posterior.
Documentación del proyecto	15%	Fuentes y PDF, URL Git, decisiones con alternativas/consecuencias, registro de commits y refinamientos del plan. Evidencia de pruebas originales y archivos de reproducción.

5.7 Plan por sprints e hitos

Cuadro 18: Hitos del módulo Placement.

Periodo	Hito	Resultado esperado
Hasta 04-09	Etapa 1	Problema, objetivos y requerimientos iniciales.
05-09 a 02-10	Etapa 2 / S2	Diseño, entorno, baseline y tráfico Nova registrados.
03-10 a 30-10	Etapa 3 / S3	Al menos 50% implementado; priorizar B07 providers/inventories, B08 traits/classes y B11 persistencia.
31-10 a 20-11	Cierre / S4	Allocations, candidates, auth y sustitución integradas y estabilizadas.
21-11 a 04-12	Evaluación / S5	Dataset comparativo, gráficos, conclusiones e informe final.

6 Conclusiones de la Etapa 2

El cierre del diseño central de S2 se sustenta en cuatro elementos terminados y en su comparación con los 26 elementos del backlog. Esta revisión incorpora vistas del reemplazo Go, modelo ER, secuencia, decisiones y trazabilidad de calidad. B21 y B16 conservan su estado formal; no se confunde la aceptación del Sprint 2 con el cierre de todo el diseño o con la implementación del reemplazo.

La etapa produjo una especificación de Placement respaldada por el servicio original: arquitectura y tablas contrastadas, alcance HTTP documentado, microversiones observadas y un flujo real de Nova que alcanzó ACTIVE y liberó recursos al eliminar la instancia.

OE1 y OE2 están cumplidos para el alcance de Sprint 2. La implementación Go, la sustitución y la evaluación de rendimiento siguen planificadas. Los fixtures pendientes y la captura de cuerpos internos deben resolverse a medida que se implemente cada contrato, conservando los límites explícitos.

El siguiente paso es desarrollar providers, inventories, traits/classes y persistencia, y compararlos con la línea base. El resultado de esta etapa habilita esa implementación sin anticipar conclusiones sobre eficiencia.

Referencias

- [1] OpenStack. *Placement API Reference*. Referencia utilizada en el proyecto y en el informe de Etapa 1. <https://docs.openstack.org/api-ref/placement/>.
- [2] OpenStack. *Nova Scheduling*. <https://docs.openstack.org/nova/latest/reference/scheduling.html>.
- [3] OpenStack. *DevStack Documentation*. <https://docs.openstack.org/devstack/latest/>.
- [4] Proyecto *modules-devstack-optimized*. Documentación y evidencias Sprint 2, capturadas el 2 de octubre de 2026. Directorios `docs/sprint2/`, `evidence/sprint2/` e `infra/`; registro de evidencia y matriz final de trazabilidad.
- [5] *Planificacion_Trabajo_Titulo_OpenStack_Placement_Glance.xlsx* y copia de seguimiento Sprint 2. Hojas Objetivos_SMART, Requerimientos, Backlog, Trazabilidad y Sprints. La copia original se mantiene inalterada.
- [6] Simon Brown. *C4 model: Component diagram*. Vista de componentes de un contenedor y sus dependencias. <https://c4model.com/diagrams/component>. Consultado el 2 de octubre de 2026.
- [7] Object Management Group. *Unified Modeling Language, versión 2.5.1*. <https://www.omg.org/spec/UML/2.5.1>. Referencia de notación de interacción; consultado el 2 de octubre de 2026.
- [8] IETF. *RFC 9110: HTTP Semantics*. Junio de 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>. Consultado el 2 de octubre de 2026.
- [9] Universidad Católica de Temuco. *Pauta y rúbrica de evaluación del curso de Trabajo de Título (INFO-1197)*. 17 de agosto de 2026. Etapa 2, página 3. Documento proporcionado para esta revisión.